

AI 기반 클라우드 아키텍처 설계 (Azure)

CONTENTS

1. AI 기반 클라우드 아키텍처 기초 설계

1. 클라우드 아키텍처 기본 및 AI 페르소나 설정
2. 네트워크·보안 설계
3. 컴퓨팅·스토리지 설계
4. 비용·고가용성·DR 설계

2. AI 기반 아키텍처 검증·문서화·운영

1. 클라우드 아키텍처 검증
2. 아키텍처 문서 자동 생성
3. 운영·모니터링 구조 설계
4. AI 기반 운영 최적화 자동화

과정 개요

- **과정명: AI 기반 클라우드 아키텍처 설계 (Azure)**
- **강의일수: 2일, 16시간**
- **강의 목표**
 - AI를 활용해 클라우드 아키텍처 설계·검증·최적화를 자동화하는 실무 능력을 확보한다.
 - 네트워크, 보안, 컴퓨팅, 스토리지 등 핵심 요소를 AI 기반으로 자동 분석·추천하여 빠르고 정확한 아키텍처 의사결정을 수행한다.
 - 향후 DevOps·IaC(Bicep/Terraform)와 연계된 운영 자동화까지 구현할 수 있는 기초를 다진다.
- **강의대상 및 사전지식**
 - 클라우드 설계·운영·DevOps·아키텍처 담당자 및 AI 기반 자동화를 업무에 적용하려는 실무자
 - 클라우드 기본 개념만 이해하면 충분하며, IaC·Kubernetes 등은 선택적 지식
- **강의구성: 총 2개의 Chapter, 8개 Section으로 구성 (이론 6시간 / 실습 10시간)**
- **실습환경: Microsoft Azure, Gen AI, Mermaid-AI**

일자별 커리큘럼 요약

- **Chapter 1. AI 기반 클라우드 아키텍처 기초 설계**
 - Section 1. 클라우드 아키텍처 기본 및 AI 페르소나 설정
 - Section 2. 네트워크·보안 설계
 - Section 3. 컴퓨팅·스토리지 설계
 - Section 4. 비용·고가용성·DR 설계
- **Chapter 2. AI 기반 아키텍처 검증·문서화·운영**
 - Section 5. 클라우드 아키텍처 검증
 - Section 6. 아키텍처 문서 자동 생성
 - Section 7. 운영·모니터링 구조 설계
 - Section 8. 운영 최적화 자동화

Chapter 1. AI 기반 클라우드 아키텍처 기초 설계

1. 클라우드 아키텍처 기본 및 AI 페르소나 설정
2. 네트워크·보안 설계
3. 컴퓨팅·스토리지 설계
4. 비용·고가용성·DR 설계

Chapter 1. AI 기반 클라우드 아키텍처 기초 설계

Section 1. 클라우드 아키텍처 기본 및 AI 페르소나 설정

학습 목표

- Azure의 핵심 서비스 구성요소(VNet, Subnet, Load Balancer 등)의 역할과 관계를 설명할 수 있다.
- MSA · Serverless · Container 아키텍처의 차이와 적용 기준을 이해한다.
- GenAI 를 활용해 비즈니스 요구사항으로부터 기본 아키텍처 구조를 도출할 수 있다.

Azure 핵심 서비스 구성요소 — 개념

- Azure 리소스는 구독(Subscription) > 리소스 그룹(Resource Group) > 개별 리소스의 계층 구조로 관리된다.
- Virtual Network(VNet)는 AWS VPC에 대응하는 논리적 격리 네트워크로, Subnet 단위로 리소스를 분리 배치한다.
- Load Balancer(L4)와 Application Gateway(L7)는 트래픽 분산 계층으로, 트래픽 특성에 따라 선택한다.

Azure 핵심 서비스 구성요소 — 상세

•핵심 구성요소

- VNet: 리전 단위 논리 네트워크, CIDR 기반 주소공간 정의, Peering으로 VNet 간 연결
- Subnet: VNet 내부의 IP 대역 분할 단위, NSG·Route Table을 Subnet 단위로 적용
- Load Balancer: 내부/공용(Public·Internal) 구성, 4계층 TCP/UDP 분산, Health Probe로 상태 점검
- Application Gateway: 7계층 HTTP(S) 라우팅, WAF(Web Application Firewall) 통합, URL 기반 라우팅 지원



3-Tier 웹 서비스 기본 아키텍처



Azure 핵심 서비스 구성요소 — 사례

• 적용 사례

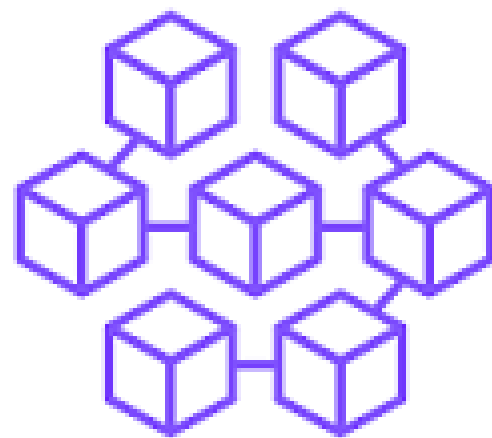
- 예시: 온라인 쇼핑몰 — Application Gateway가 도메인별 트래픽을 Web/API Subnet으로 분기하고, 내부 Load Balancer가 각 Subnet 내 VM 간 부하를 분산한다.
- Azure Portal에서 리소스 그룹 단위로 태그(Tag)를 부여하면 이후 비용·운영 관리가 용이해진다.



MSA · Serverless · Container 아키텍처 개요 — 개념

• MSA · Serverless · Container 아키텍처 개요

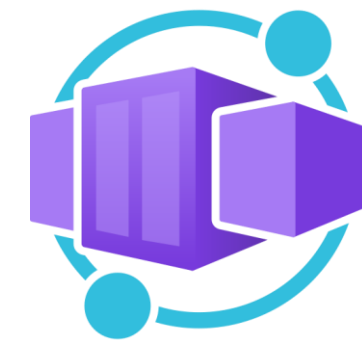
- 모놀리식(Monolithic) 대비 MSA(Microservices Architecture)는 서비스 단위 독립 배포 · 확장이 가능하다.
- Serverless(Azure Functions)는 이벤트 기반 실행으로 인프라 관리 부담을 최소화한다.
- Container(AKS, Azure Container Apps)는 이식성과 일관된 실행 환경을 제공한다.



MSA · Serverless · Container 아키텍처 개요 — 상세

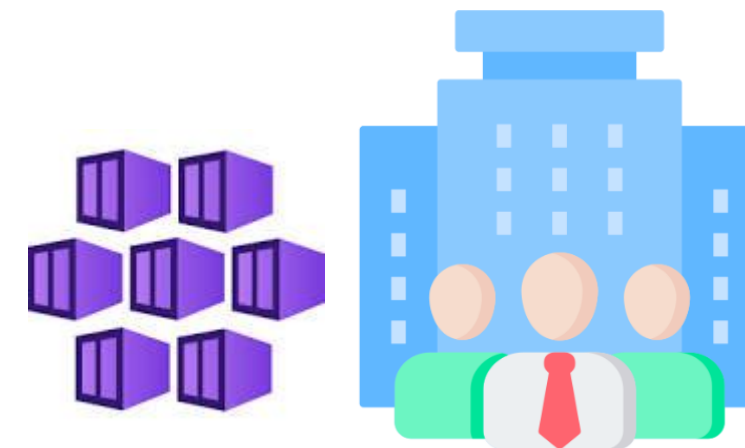
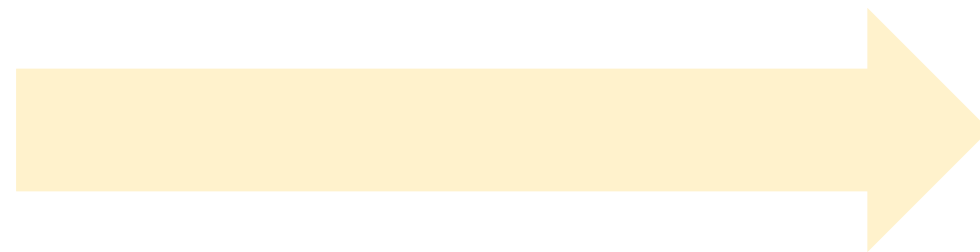
• 핵심 구성요소

- Azure Functions: HTTP · Timer · Queue 트리거 지원,
소비 계획(Consumption Plan) 기반 자동 스케일 및 종량 과금
- AKS(Azure Kubernetes Service): 관리형 Kubernetes, 노드 풀 단위 확장, Helm/GitOps 연계
- Azure Container Apps: Kubernetes 지식 없이 컨테이너 마이크로서비스 운영,
KEDA 기반 이벤트 스케일링
- 선택 기준: 트래픽 예측 가능성, 실행 시간, 상태 유지 여부, 팀의 운영 역량을 종합 고려



MSA · Serverless · Container 아키텍처 개요 — 실무 팁

- 스타트업 초기에는 Container Apps/Functions로 빠르게 출시한 뒤, 트래픽 증가 시 AKS로 전환하는 단계적 접근이 일반적이다.



요구사항 입력 → 기본 아키텍처 구조 도출 (AI 활용)

- Gen AI 에 비즈니스 요구사항(트래픽량, 가용성 목표, 예산, 규정 준수 등)을 구조화하여 입력하면, 후보 아키텍처 초안을 빠르게 얻을 수 있다.
- AI 산출물은 초안이며, 반드시 아키텍트의 검증과 조정을 거쳐야 한다.



AI 활용 아키텍처 도출 절차 예시

- Step 1. 요구사항 정리:
사용자 수, 트래픽 패턴, RTO/RPO, 보안·컴플라이언스 요건 문서화
- Step 2. 프롬프트 작성:
서비스 목적, 제약조건, 우선순위(비용/성능/가용성)를 명시하여 입력
- Step 3. 초안 검토:
제안된 서비스 조합 (VNet, Compute, Storage, DB)을 Well-Architected 기준으로 검증
- Step 4. 반복 개선:
부족한 요소(보안, 모니터링 등)를 추가 질의하며 완성도를 높임

컴퓨팅 모델 비교 — VM vs AKS vs Azure Functions

항목	Azure VM	AKS (컨테이너)	Azure Functions (서버리스)
관리 부담	높음 (OS·패치 직접 관리)	중간 (클러스터는 관리형)	매우 낮음 (인프라 완전 위임)
확장 방식	VMSS 수동/자동 스케일	노드 풀·Pod 오토스케일	트리거 기반 자동 스케일(0까지 축소)
과금 방식	가동 시간 기준	노드 가동 시간 기준	실행 횟수·시간 기준(종량제)
적합 워크로드	상태 유지·레거시 애플리케이션	다수 마이크로서비스, 이식성 필요	이벤트 처리, 짧은 비동기 작업

출처: Microsoft Learn — Choose an Azure compute service

공식문서 참고

- **Azure Virtual Network 개요**

- <https://learn.microsoft.com/en-us/azure/virtual-network/virtual-networks-overview>
- VNet의 기본 개념과 구성요소

- **Azure 애플리케이션을 위한 10가지 설계 원칙**

- <https://learn.microsoft.com/en-us/azure/architecture/guide/design-principles/>
- Well-Architected 설계 원칙의 기반 문서

- **Azure Well-Architected Framework**

- <https://learn.microsoft.com/en-us/azure/well-architected/>
- 설계·검증의 공식 프레임워크 허브

실습 Lab

- **실습 시나리오**

- 월 방문자 50만 명 규모의 이커머스 서비스에 대한 기본 Azure 아키텍처 초안을 Gen AI로 도출한다.

- **Gen AI 프롬프트 예시**

- “너는 Azure 클라우드 아키텍트다. 월간 방문자 50만 명, 피크 시간대 트래픽 집중, 예산은 월 500만원 이 내인 이커머스 웹서비스의 기본 아키텍처를 VNet, Subnet, 컴퓨팅, 스토리지, DB 구성 중심으로 제안해 줘. 고가용성과 비용 효율을 함께 고려해줘.”

- **점검 포인트**

- 제안된 구성요소가 요구사항(트래픽, 예산, 가용성)을 충족하는지 확인
- 누락된 보안·모니터링 요소가 없는지 점검
- Azure 서비스명과 역할이 정확히 매칭되는지 검증

핵심 요약

- VNet/Subnet: 네트워크 격리 및 분할의 기본 단위
- Load Balancer/Application Gateway: 계층별 트래픽 분산 전략
- MSA/Serverless/Container: 서비스 특성에 맞는 컴퓨팅 모델 선택
- AI 활용: 요구사항 기반 아키텍처 초안 자동 생성으로 설계 시간 단축
- 검증 필수: AI 산출물은 항상 Well-Architected 관점에서 재검토

Chapter 1. AI 기반 클라우드 아키텍처 기초 설계

Section 2. 네트워크·보안 설계

학습 목표

- VNet 설계 패턴과 서브넷 구성 전략을 수립할 수 있다.
- Azure AD(Entra ID) 기반 인증과 NSG 기반 트래픽 제어 원리를 이해한다.
- 요구조건으로부터 AI를 활용해 보안정책안을 자동 생성할 수 있다.

VNet 설계 패턴 / 서브넷 구성

- VNet 주소공간은 향후 확장(Peering, 온프레미스 연결)을 고려해 충분히 여유 있게 설계한다.
- Public/Private Subnet 분리로 인터넷 노출 리소스를 최소화하는 것이 기본 원칙이다.

VNet 설계 패턴 / 서브넷 구성

- 핵심 구성요소

- Hub-Spoke 패턴:

- 중앙 Hub VNet에 공통 서비스(방화벽, VPN Gateway) 배치, Spoke VNet에서 워크로드 분리 운영

- 3-Tier Subnet:

- Web/App/Data Subnet으로 분리, Subnet 간 트래픽은 NSG로 통제

- VNet Peering:

- 동일·타 리전 VNet 간 저지연 연결, 전이적(Transitive) 라우팅 미지원에 유의

- Private Endpoint:

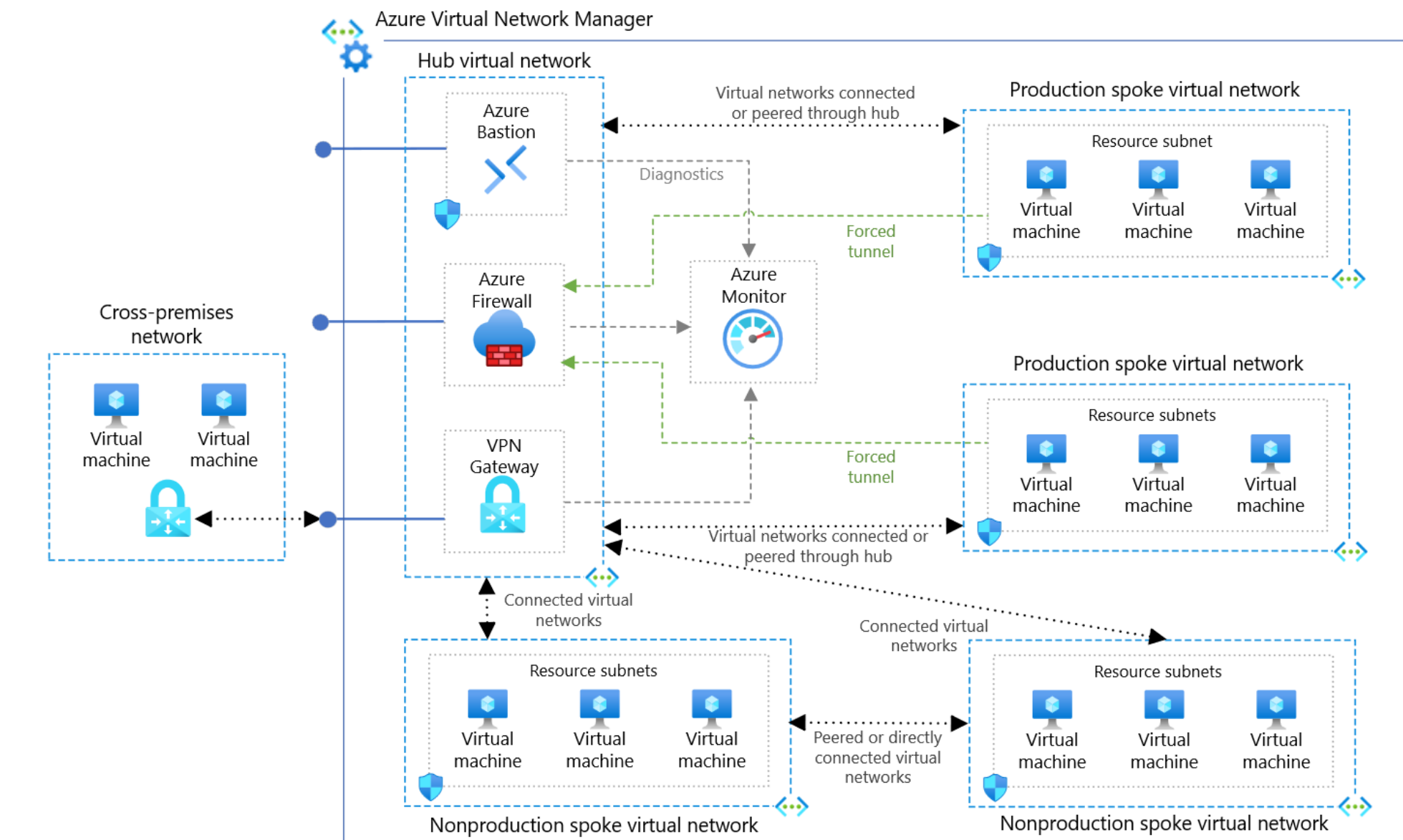
- PaaS 서비스(Storage, SQL DB 등)를 VNet 내부 사설 IP로 연결해 인터넷 노출 차단

VNet 내부 네트워크·보안 구성 흐름



VNet 설계 패턴 / 서브넷 구성 — 사례

- Hub-Spoke 구조에서 Azure Firewall을 Hub에 배치하면 모든 Spoke의 아웃바운드 트래픽을 중앙에서 통제할 수 있다.



Azure AD(Entra ID), NSG 보안 정책 원리

- Azure AD(Entra ID)는 신원(Identity) 관리의 중심으로, RBAC(역할 기반 액세스 제어)와 결합해 최소 권한 원칙을 구현한다.
- NSG(Network Security Group)는 Subnet·NIC 단위로 인바운드/아웃바운드 트래픽을 필터링하는 상태 저장(Stateful) 방화벽이다.



Azure AD(Entra ID), NSG 보안 정책 원리 — 상세

• 핵심 구성요소

- Azure RBAC:

구독/리소스 그룹/리소스 범위(Scope)별로 소유자·기여자·읽기 권한 등 역할 할당

- 조건부 액세스(Conditional Access):

위치, 기기 상태, 위험도 기반 로그인 정책 적용

- NSG 규칙:

우선순위(Priority) 숫자가 낮을수록 먼저 적용, 5-tuple 기반 필터링

- Azure Firewall/WAF:

NSG보다 상위 계층에서 SQL Injection, XSS 등 애플리케이션 수준 위협 방어

Azure AD(Entra ID), NSG 보안 정책 원리 — 실무 팁

- AWS의 Security Group(인스턴스 단위)과 달리 Azure NSG는 Subnet과 NIC 두 곳에 모두 적용할 수 있으므로 규칙 중복·충돌에 주의해야 한다.

요구조건 입력 → 보안정책안 자동 생성 — 개념

- 보안 요구사항(허용 포트, 접근 주체, 컴플라이언스 기준)을 구조화하면 AI가 NSG 규칙 초안과 RBAC 역할 매핑안을 제시할 수 있다.

요구조건 입력 → 보안정책안 자동 생성 — 절차

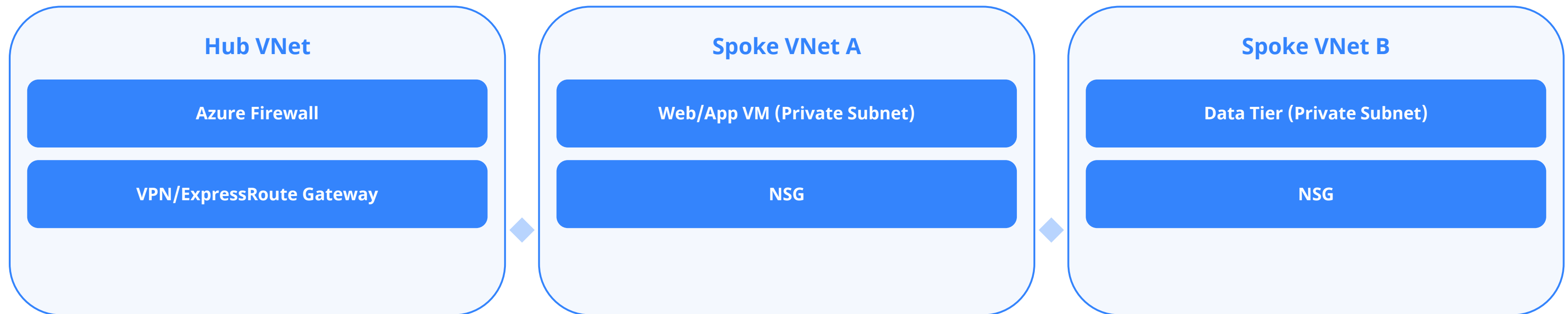
- Step 1. 자산 분류: Subnet별 리소스 유형과 민감도 등급 정리
- Step 2. 요구조건 입력: 허용 포트/프로토콜, 접근 주체(팀, 서비스)를 Gen AI에 전달
- Step 3. 초안 생성: NSG 규칙 목록(우선순위, 방향, 포트, 소스) 및 RBAC 역할 할당안 도출
- Step 4. 최소권한 검증: 불필요하게 광범위한 규칙(0.0.0.0/0 등)이 없는지 재검토 후 적용

NSG 기본 보안 규칙 (인바운드)

우선순위	규칙 이름	동작	설명
65000	AllowVnetInBound	허용	동일 VNet 내부 트래픽 허용
65001	AllowAzureLoadBalancerInBound	허용	Azure Load Balancer Health Probe 트래픽 허용
65500	DenyAllInBound	차단	그 외 모든 인바운드 트래픽 차단

출처: Microsoft Learn — Network security groups overview · 우선순위 숫자가 낮을수록 먼저 적용되며, 기본 규칙은 삭제할 수 없고 더 높은 우선순위 규칙으로만 재정의할 수 있습니다.

Hub-Spoke 네트워크 토폴로지



Spoke VNet은 Peering으로 Hub VNet과 연결되며, 모든 아웃바운드·인터넷 트래픽은 Hub의 Azure Firewall을 경유해 중앙에서 통제합니다.

공식문서 참고

- **Network security groups 개요**

- <https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>
- NSG 규칙 구조와 기본 규칙 정의

- **Hub-spoke network topology**

- <https://learn.microsoft.com/en-us/azure/architecture/networking/architecture/hub-spoke>
- 허브-스포크 참조 아키텍처

- **Azure networking services 개요**

- <https://learn.microsoft.com/en-us/azure/networking/networking-overview>
- Azure 네트워킹 서비스 전체 지도

실습 Lab

• 실습 시나리오

- 3-Tier 아키텍처(Web/App/Data Subnet)에 대한 NSG 규칙안을 Gen AI로 도출한다.

• Gen AI 프롬프트 예시

- “Azure 3-Tier 아키텍처(Web/App/Data Subnet)에 대해 각 Subnet의 NSG 인바운드/아웃바운드 규칙을 표 형태로 제안해줘. Web Subnet은 인터넷에서 443만 허용, App Subnet은 Web Subnet에서만 접근 허용, Data Subnet은 App Subnet에서만 접근을 허용하는 최소권한 원칙을 적용해줘.”

• 점검 포인트

- 각 Subnet 간 접근이 최소권한 원칙을 지키는지 확인
- 우선순위 충돌이나 중복 규칙이 없는지 검토
- 관리용 포트(RDP/SSH)가 인터넷에 열려있지 않은지 확인

핵심 요약

- Hub-Spoke/3-Tier: 대표적인 VNet 설계 패턴
- Private Endpoint: PaaS 서비스의 사설 연결로 노출 최소화
- Azure AD + RBAC: 신원 기반 최소권한 접근 제어의 핵심
- NSG: Subnet/NIC 단위 상태 저장 트래픽 필터링
- AI 활용: 요구조건 기반 보안정책 초안 생성으로 검토 시간 단축

Chapter 1. AI 기반 클라우드 아키텍처 기초 설계

Section 3. 컴퓨팅·스토리지 설계

학습 목표

- Azure VM, VMSS, Blob Storage, Azure SQL Database의 특징과 활용 전략을 이해한다.
- 성능과 비용을 함께 고려한 스펙 산정 원리를 익힌다.
- 트래픽 기준으로 컴퓨팅 스펙을 산정할 수 있다.

Azure VM/VMSS/Blob Storage/Azure SQL Database 구성 전략 — 개념

- Azure VM은 다양한 시리즈(범용 D, 컴퓨팅 최적화 F, 메모리 최적화 E 등)로 워크로드에 맞게 선택한다.
- VMSS(Virtual Machine Scale Sets)는 동일 구성의 VM을 자동으로 확장·축소하는 서비스로 AWS ASG에 대응한다.

Azure VM/VMSS/Blob Storage/Azure SQL Database

• 핵심 구성요소

- VM 시리즈:

웹서버(D시리즈, 범용), 배치·연산(F시리즈, 컴퓨팅 최적화), DB(E시리즈, 메모리 최적화)

- VMSS: Scale-out 규칙(CPU/메모리/큐 길이 기준),

Availability Zone 균등 분산 배치 지원

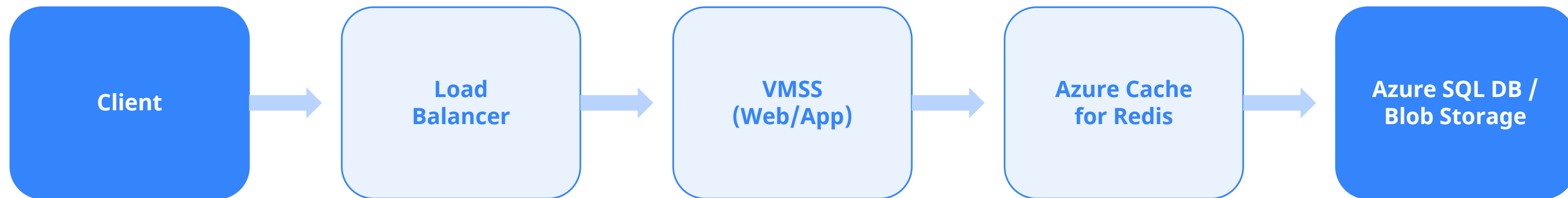
- Blob Storage: Hot/Cool/Archive 계층으로 접근 빈도별 비용 최적화,

LRS/ZRS/GRS 중복 옵션으로 내구성 선택

- Azure SQL Database: DTU/vCore 두 가지 과금 모델, 자동 튜닝·자동 백업 내장,

Hyperscale로 대용량 확장 지원

컴퓨팅·스토리지 구성 흐름



Azure VM/VMSS/Blob Storage/Azure SQL Database 구성 전략 — 사례

- 이미지 업로드가 많은 서비스는 수명 주기 관리 정책으로 최근 파일은 Hot 계층에, 오래된 파일은 Cool/Archive 계층으로 자동 이동해 스토리지 비용을 절감한다.

성능&비용 기준 스펙 산정 원리 — 개념

- 스펙 산정은 예상 동시 사용자 수,
요청당 리소스 사용량, 목표 응답시간을 기준으로 역산한다.

성능&비용 기준 스펙 산정 원리 — 상세

• 핵심 구성요소

- CPU 산정: $(\text{초당 요청 수} \times \text{요청당 평균 CPU 처리시간}) \div \text{목표 CPU 사용률}$
(예: 70%)로 필요 vCore 산출
- 메모리 산정: $\text{세션·캐시 데이터 크기} \times \text{동시 접속자 수} + \text{여유분}(20\sim30\%)$
- 스토리지 IOPS:
DB 트랜잭션 처리량(TPS) 기준으로 Premium SSD 등 필요 등급의 Managed Disk 선택
- 비용 최적화: 예약 인스턴스(Reserved VM Instances), Azure Hybrid Benefit,
Spot VM 활용으로 최대 72%까지 절감

성능&비용 기준 스펙 산정 원리 — 실무 팁

- Azure Pricing Calculator와 Azure Advisor를 병행 활용하면 스펙 산정과 비용 검증을 동시에 수행할 수 있다.

트래픽 기준 스펙 산정 — 개념

- 실측 트래픽 패턴(피크/평시 비율)을 기반으로 오토스케일 임계값을 설정해야
과다·과소 프로비저닝을 방지할 수 있다.

트래픽 기준 스펙 산정 — 절차

- Step 1. 트래픽 프로파일 수집: 시간대별·요일별 요청 수, 피크 대비 평시 비율 분석
- Step 2. 인스턴스당 처리량 측정: 부하 테스트로 VM 1대가 처리 가능한 초당 요청 수(RPS) 확인
- Step 3. 필요 인스턴스 수 계산: $\text{피크 RPS} \div \text{인스턴스당 RPS} \times \text{여유율}(1.2\sim 1.3)$
- Step 4. VMSS 오토스케일 규칙 설정: CPU 70% 초과 시 확장, 30% 미만 시 축소 등 임계값 정의

Azure Storage 중복(redundancy) 옵션 비교

옵션	복제 방식	연간 내구성	특징
LRS	동일 데이터센터 내 3중 복제	99.999999999% (11개 9)	최저 비용, 데이터센터 재해에는 취약
ZRS	3개 이상 가용영역에 복제	99.999999999% (12개 9)	가용영역 장애에도 읽기/쓰기 유지
GRS	LRS + 보조 리전 비동기 복제	99.99999999999999% (16개 9)	리전 재해 대비, 보조 리전은 기본 읽기 불가
RA-GRS	GRS + 보조 리전 읽기 허용	99.99999999999999% (16개 9)	리전 장애 시에도 보조 리전에서 읽기 가능

출처: Microsoft Learn — Azure Storage redundancy

Azure SQL Database 구매 모델 비교

항목	DTU 모델	vCore 모델
과금 단위	CPU·메모리·IO를 묶은 DTU	vCPU·메모리를 개별 산정
티어	Basic / Standard / Premium	프로비저닝 / 서버리스
서버리스	미지원	사용량 기반 자동 일시중지·재개 지원
적합 대상	단순하고 예측 가능한 소규모 워크로드	세밀한 스펙 조정이 필요한 중대형 워크로드

출처: Microsoft Learn — Azure SQL Database purchasing models

공식문서 참고

- **Sizes for virtual machines in Azure**

- <https://learn.microsoft.com/en-us/azure/virtual-machines/sizes>
- VM 시리즈별 스펙과 용도

- **Access tiers for blob data**

- <https://learn.microsoft.com/en-us/azure/storage/blobs/access-tiers-overview>
- Hot/Cool/Cold/Archive 계층 정의

- **Azure SQL Database purchasing models**

- <https://learn.microsoft.com/en-us/azure/azure-sql/database/purchasing-models>
- DTU·vCore 모델 공식 비교

실습 Lab

• 실습 시나리오

- 피크 시간대 초당 1,000건 요청을 처리해야 하는 서비스의 VMSS 스펙을 Gen AI로 산정한다.

• Gen AI 프롬프트 예시

- “Azure VMSS로 피크 시간대 초당 1,000건 HTTP 요청을 처리해야 하는 웹 서비스가 있다. VM 1대(D2s v5)가 초당 80건을 처리할 수 있다고 가정할 때 필요한 최소/최대 인스턴스 수와 오토스케일 규칙(CPU 기준)을 제안해줘.”

• 점검 포인트

- 계산된 인스턴스 수가 여유율을 포함하는지 확인
- 오토스케일 확장/축소 임계값이 합리적인지 검토
- 비용 절감을 위한 예약 인스턴스/Spot VM 적용 가능 여부 확인

핵심 요약

- VM 시리즈: 워크로드 특성별 D/F/E 시리즈 선택
- VMSS: 오토스케일 기반 컴퓨팅 확장의 핵심
- Blob Storage 계층화: Hot/Cool/Archive로 비용 최적화
- Azure SQL Database: DTU/vCore 모델과 Hyperscale 확장
- 스펙 산정: 트래픽·성능·비용을 함께 고려한 역산 접근

Chapter 1. AI 기반 클라우드 아키텍처 기초 설계

Section 4. 비용·고가용성·DR 설계

학습 목표

- Availability Zone/Set 기반 고가용성 설계 원칙을 이해한다.
- DR Tier 개념과 Azure Site Recovery 기반 재해복구 전략을 수립할 수 있다.
- 기존 아키텍처를 입력해 AI로 비용 절감안을 도출할 수 있다.

High Availability 설계 원칙 — 개념

- 고가용성은 단일 장애점(SPOF)을 제거하는 것에서 시작하며, Azure는 리전 내 Availability Zone과 Availability Set 두 메커니즘을 제공한다.

High Availability 설계 원칙 — 상세

• 핵심 구성요소

- Availability Zone(AZ): 리전 내 물리적으로 분리된 데이터센터 그룹, 전원·냉각·네트워크가 독립되어 존 단위 장애에 대응
- Availability Set: 동일 데이터센터 내 장애 도메인·업데이트 도메인으로 VM 분산 배치
- Zone-redundant 서비스: Load Balancer(Standard), Azure SQL Database, Storage(ZRS) 등이 AZ 간 자동 이중화 지원
- SLA 조합: 다중 AZ 배포 시 VM SLA가 99.9%에서 99.99%로 향상

리전 간 고가용성·DR 구성



High Availability 설계 원칙 — 사례

- 3개 AZ에 각각 VMSS 인스턴스를 분산 배치하고 Standard Load Balancer로 트래픽을 분산하면 특정 AZ 장애 시에도 서비스를 지속할 수 있다.

DR Tier 및 재해복구 전략 — 개념

- DR 전략은 RTO(복구 목표 시간)와 RPO(복구 목표 시점) 요구사항에 따라 Tier를 구분해 설계한다.

DR Tier 및 재해복구 전략 — 상세

• 핵심 구성요소

- Backup & Restore: 가장 저비용, RTO/RPO가 가장 길다(수 시간~수일)
- Pilot Light: 최소 구성만 상시 유지, 장애 시 스케일업하여 전환(RTO 수십 분)
- Warm Standby: 축소된 운영 환경을 상시 가동, 장애 시 스케일 확장(RTO 수 분)
- Multi-Site(Active-Active): 두 리전에서 동시 서비스, RTO/RPO가 거의 0에 근접(비용 최고)
- Azure Site Recovery(ASR): VM 단위 리전 간 복제 및 장애 조치(Failover) 자동화, Paired Region 활용 권장

DR Tier 및 재해복구 전략 — 실무 팁

- Azure는 리전을 Paired Region으로 묶어 순차 업데이트·지리적 이중화를 지원하므로 DR 대상 리전 선정 시 우선 고려한다.

기존 아키텍처 입력 → 비용 절감안 자동 생성 — 개념

- 현재 아키텍처 구성과 사용 패턴을 AI에 입력하면 유휴 리소스, 과대 프로비저닝, 요금제 최적화 기회를 빠르게 식별할 수 있다.

기존 아키텍처 입력 → 비용 절감안 자동 생성 — 절차

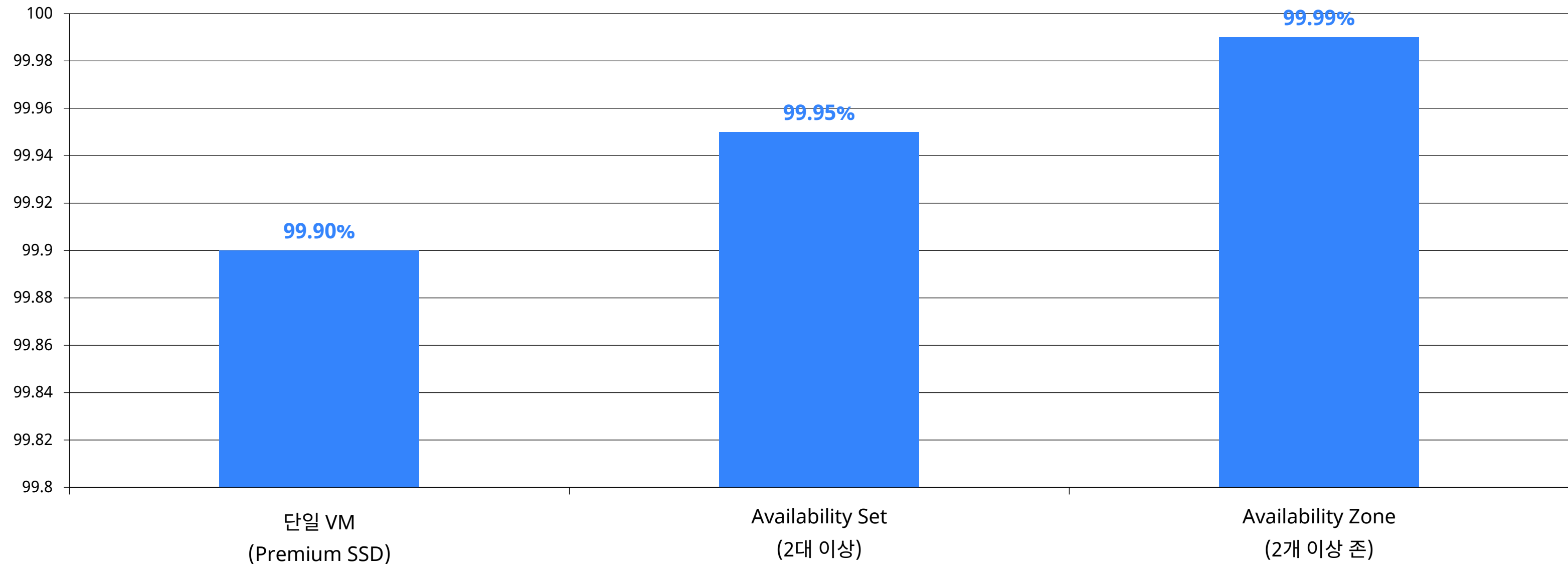
- Step 1. 현황 정리: 리소스 목록, 사용률(CPU/메모리), 현재 요금제(온디맨드/예약) 문서화
- Step 2. AI 분석 요청: Gen AI/Azure Advisor에 리소스 사용 패턴을 입력해 절감 기회 도출
- Step 3. 절감안 검토: 예약 인스턴스 전환, 유휴 리소스 삭제, 스토리지 계층 조정, Right-sizing 등 항목별 절감액 산정
- Step 4. 우선순위 적용: 서비스 영향도가 낮은 항목부터 단계적으로 적용

Azure Site Recovery RPO/RTO 참고값

항목	값	설명
RTO SLA	약 1시간	Azure-to-Azure 복제 기준 공식 SLA
RPO	약 5분	일반적인 구성 기준 실측 수준
최소 복제 주기	약 30초	Azure VM 간 복제 설정 가능한 최소 간격

출처: Microsoft Learn — Azure Site Recovery FAQ (azure-to-azure-common-questions)

가용성 구성별 SLA 비교



출처: Microsoft Azure SLA for Virtual Machines

공식문서 참고

- **Azure Reliability documentation**

- <https://learn.microsoft.com/en-us/azure/reliability/overview>
- 가용영역 · 리전 복원력 공식 가이드

- **About Azure Site Recovery**

- <https://learn.microsoft.com/en-us/azure/site-recovery/site-recovery-overview>
- ASR 개념과 아키텍처

- **Azure-to-Azure 복제 FAQ**

- <https://learn.microsoft.com/en-us/azure/site-recovery/azure-to-azure-common-questions>
- RPO/RTO 등 실무 질의응답

실습 Lab

• 실습 시나리오

- 상시 가동 중인 D4s v5 VM 10대(평균 CPU 사용률 15%)에 대한 비용 절감안을 Gen AI로 도출한다.

• Gen AI 프롬프트 예시

- “Azure에서 D4s v5 VM 10대를 24시간 상시 운영 중이며 평균 CPU 사용률이 15%다. Right-sizing, 예약 인스턴스, Azure Hybrid Benefit 관점에서 비용 절감 방안을 구체적인 예상 절감률과 함께 제안해줘.”

• 점검 포인트

- 제안된 절감안이 성능·가용성에 미치는 영향을 검토했는지 확인
- Right-sizing 이후 목표 사용률(예: 60~70%)이 적절한지 검증
- 절감안 적용 우선순위가 리스크 대비 효과 순으로 정렬되었는지 확인

핵심 요약

- Availability Zone/Set: 리전 내 고가용성 확보의 핵심 메커니즘
- DR Tier: Backup&Restore ~ Multi-Site까지 RTO/RPO 기준으로 선택
- Azure Site Recovery: 리전 간 자동 복제·장애조치 지원
- 비용 절감: Right-sizing, 예약 인스턴스, Hybrid Benefit이 대표 수단
- AI 활용: 기존 구성 입력만으로 절감 기회를 빠르게 식별

Chapter 2. AI 기반 아키텍처 검증·문서화·운영

1. 클라우드 아키텍처 검증
2. 아키텍처 문서 자동 생성
3. 운영·모니터링 구조 설계
4. AI 기반 운영 최적화 자동화

Chapter 2. AI 기반 아키텍처 검증·문서화·운영

Section 1. 클라우드 아키텍처 검증

학습 목표

- 아키텍처 검증을 위한 체크리스트 설계 방법을 이해한다.
- AI를 활용해 취약점과 병목을 자동으로 탐지하는 접근법을 익힌다.
- 설계 입력값으로부터 개선 포인트를 자동 도출할 수 있다.

검증 체크리스트 설계 방법 — 개념

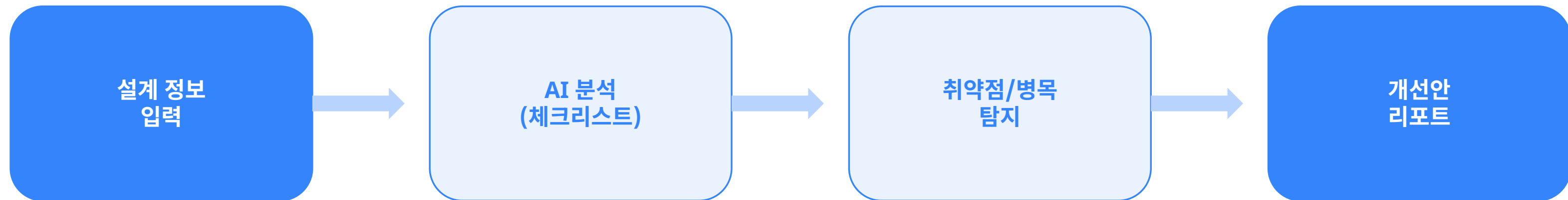
- Azure Well-Architected Framework의 5대 기둥
(비용 최적화, 운영 우수성, 성능 효율성, 안정성, 보안)을 기준으로 체크리스트를 구성한다.

검증 체크리스트 설계 방법 — 상세

• 핵심 구성요소

- 안정성(Reliability): SPOF 제거 여부, 백업·복구 계획, Multi-AZ 구성 여부
- 보안(Security): 최소권한 적용, 데이터 암호화(전송/저장), Private Endpoint 사용 여부
- 성능 효율성: 적정 SKU 선택, 캐싱 전략, Auto-scale 설정 여부
- 비용 최적화 / 운영 우수성: 유휴 리소스·예약 인스턴스 활용률, IaC 적용 및 모니터링 체계 구축 여부

AI 기반 아키텍처 검증 파이프라인



검증 체크리스트 설계 방법 — 사례

- 체크리스트는 Yes/No 문항이 아닌 근거(Evidence)를 함께 기록하도록 설계해야 실효성이 높다.

AI 기반 취약점/병목 자동 탐지 — 개념

- 아키텍처 다이어그램이나 구성 정보를 AI에 입력하면 알려진 안티패턴 (단일 인스턴스, 개방된 포트 등)을 빠르게 스캐닝할 수 있다.

AI 기반 취약점/병목 자동 탐지 — 상세

• 핵심 구성요소

- 대표 안티패턴: 단일 VM으로만 운영되는 프로덕션 서비스, 0.0.0.0/0 전체 허용 NSG 규칙, 백업 미설정 DB
- 성능 병목 패턴: DB 커넥션 풀 고갈, 캐시 미적용으로 인한 반복 쿼리, 동기 호출 체인으로 인한 지연 누적
- Azure 네이티브 도구 병행: Azure Advisor(권고안), Microsoft Defender for Cloud(보안 점수)와 AI 분석을 상호 보완적으로 활용

AI 기반 취약점/병목 자동 탐지 — 실무 팁

- AI 탐지 결과는 오탐(False Positive) 가능성이 있으므로 실제 리소스 구성과 반드시 교차 검증한다.

입력된 설계 → 개선 포인트 자동 도출 — 개념

- 아키텍처 구성 정보를 구조화해 AI에 입력하면 우선순위가 매겨진 개선안을 얻을 수 있다.

입력된 설계 → 개선 포인트 자동 도출 — 절차

- Step 1. 설계 정보 정리: 리소스 목록, 네트워크 구성, 트래픽 패턴을 텍스트·표로 정리
- Step 2. AI 검증 요청: Well-Architected 5대 기둥 기준으로 리뷰 요청
- Step 3. 개선안 우선순위화: 영향도(고/중/저) × 조치 난이도로 매트릭스 작성
- Step 4. 개선 로드맵 수립: 즉시 조치, 단기(1개월), 중장기(분기) 항목으로 구분

Azure Well-Architected Framework 5대 기둥 (공식)

기둥	핵심 관심사	적용 원칙
Reliability (안정성)	복원력, 가용성, 복구	비즈니스 요구사항 기반 설계, 단순성 유지
Security (보안)	데이터 보호, 위협 탐지·완화	기밀성·무결성·가용성 보호
Cost Optimization (비용 최적화)	비용 모델링, 예산, 낭비 절감	사용량·요금제 최적화
Operational Excellence (운영 우수성)	통합 관측성, DevOps 관행	표준화·모니터링·안전한 배포
Performance Efficiency (성능 효율성)	확장성, 부하 테스트	수평 확장, 지속적 테스트·모니터링

출처: Microsoft Learn — Microsoft Azure Well-Architected Framework pillars

공식문서 참고

- **Well-Architected Framework pillars**

- <https://learn.microsoft.com/en-us/azure/well-architected/pillars>
- 5대 기둥 공식 정의

- **Introduction to Azure Advisor**

- <https://learn.microsoft.com/en-us/azure/advisor/advisor-overview>
- AI 기반 권고 서비스 개요

- **Microsoft Defender for Cloud 개요**

- <https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-cloud-introduction>
- 보안 상태 점수·취약점 관리

실습 Lab

• 실습 시나리오

- 단일 VM + 단일 리전 DB로 구성된 현재 아키텍처를 Gen AI로 검증한다.

• Gen AI 프롬프트 예시

- “다음 Azure 아키텍처를 Well-Architected Framework 5대 기둥 기준으로 검토해줘: 단일 VM(D2s v5) 1대에서 웹서버 운영, 백업 미설정 Azure SQL Database 1대, 모든 NSG 규칙이 0.0.0.0/0 허용. 발견된 리스크와 개선안을 영향도 순으로 정리해줘.”

• 점검 포인트

- 안정성(SPOF), 보안(개방 규칙), 운영(백업 미설정) 리스크가 모두 식별되었는지 확인
- 개선안이 영향도·난이도 기준으로 우선순위화되었는지 검토
- 즉시 조치가 필요한 항목(보안 관련)이 명확히 구분되었는지 확인

핵심 요약

- Well-Architected Framework: 5대 기둥 기반 체계적 검증
- 안티패턴 탐지: 단일 인스턴스, 개방 규칙, 백업 미설정 등 반복 확인
- AI+네이티브 도구 병행: Advisor·Defender for Cloud와 상호 보완
- 개선 우선순위화: 영향도 × 난이도 매트릭스로 로드맵 수립
- 교차 검증: AI 결과는 반드시 실제 구성과 대조

Chapter 2. AI 기반 아키텍처 검증·문서화·운영

Section 2. 아키텍처 문서 자동 생성

학습 목표

- HLD(High-Level Design) 문서의 작성 방식과 구성요소를 이해한다.
- Mermaid-AI를 활용해 Infra Diagram과 Sequence Diagram을 자동 생성할 수 있다.
- Use Case 입력으로부터 아키텍처 다이어그램을 생성할 수 있다.

HLD(High-Level Design) 작성 방식 — 개념

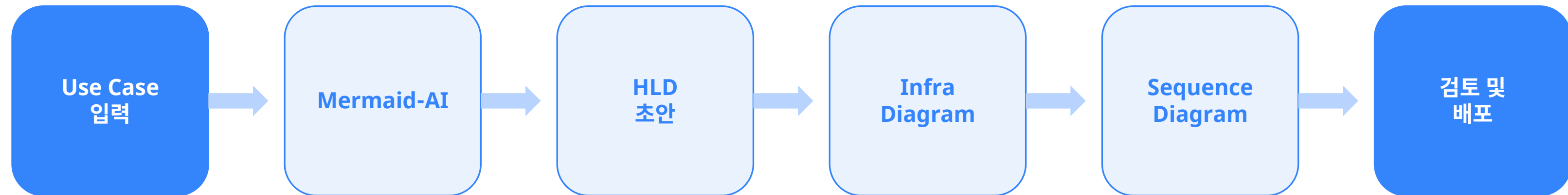
- HLD는 시스템 전체 구조를 이해관계자가 빠르게 파악하도록 돕는 상위 수준 설계 문서다.

HLD(High-Level Design) 작성 방식 — 상세

• 핵심 구성요소

- 구성요소: 시스템 개요, 아키텍처 다이어그램, 주요 컴포넌트 역할, 데이터 흐름, 비기능 요구사항(성능/보안/가용성)
- LLD(Low-Level Design)와의 차이: HLD는 '무엇을·왜'에 집중, LLD는 '어떻게(구현 상세)'에 집중
- 좋은 HLD의 조건: 비개발자도 이해 가능한 수준의 추상화, 다이어그램과 텍스트의 상호 보완

AI 기반 아키텍처 문서 자동화 흐름



HLD(High-Level Design) 작성 방식 — 사례

- HLD 문서 표준 목차:
 - 1) 개요
 - 2) 아키텍처 다이어그램
 - 3) 컴포넌트 설명
 - 4) 데이터 흐름
 - 5) 보안/가용성 고려사항
 - 6) 용어집

Infra Diagram · Sequence Diagram 자동 생성 — 개념

- Mermaid-AI는 텍스트 기반 설명을 다이어그램 문법(Mermaid syntax)으로 변환해주는 도구로, 문서화 속도를 크게 높인다.

Infra Diagram · Sequence Diagram 자동 생성 — 상세

• 핵심 구성요소

- Infra Diagram: 아키텍처 구성요소(VNet, VM, DB 등)와 연결 관계를 노드·엣지로 표현
- Sequence Diagram: 사용자 요청부터 응답까지 컴포넌트 간 호출 순서와 시간 흐름을 표현
- Mermaid 문법: graph TD(플로우차트), sequenceDiagram(시퀀스) 등 각 노드에 서비스명·역할 라벨링
- 유지보수 이점: 텍스트 기반이므로 Git으로 버전 관리 가능, 아키텍처 변경 시 다이어그램도 함께 업데이트 용이

Infra Diagram · Sequence Diagram 자동 생성 — 실무 팁

- 다이어그램은 한 화면에 담을 정보량을 15개 노드 이내로 제한해야 가독성이 유지된다.

Use Case 입력 → 아키텍처 다이어그램 생성 — 개념

- 사용자 시나리오(Use Case)를 순서대로 서술하면 AI가 이를 Sequence Diagram과 Infra Diagram으로 변환할 수 있다.

Use Case 입력 → 아키텍처 다이어그램 생성 — 절차

- Step 1. Use Case 서술: '사용자가 로그인 후 상품을 주문한다'와 같이 행위 중심으로 단계 작성
- Step 2. Mermaid-AI 프롬프트 작성: 각 단계에 관여하는 Azure 서비스와 순서를 명시
- Step 3. 다이어그램 생성 및 검토: 생성된 Mermaid 코드를 렌더링하여 누락된 컴포넌트 확인
- Step 4. 문서 통합: 생성된 다이어그램을 HLD 문서에 삽입하고 텍스트 설명 보완

HLD vs LLD 비교

항목	HLD (상위설계)	LLD (상세설계)
목적	전체 구조 이해	구현 상세 정의
대상 독자	경영진·이해관계자·아키텍트	개발자·엔지니어
포함 내용	아키텍처 다이어그램, 데이터 흐름	API 명세, 스키마, 알고리즘
변경 빈도	낮음	상대적으로 높음

Mermaid Sequence Diagram 코드 예시

```
sequenceDiagram
    participant User
    participant AppGW as Application Gateway
    participant App as App Service
    participant AAD as Azure AD B2C
    participant DB as Azure SQL Database

    User->>AppGW: 로그인 요청
    AppGW->>App: 요청 전달
    App->>AAD: 인증 위임
    AAD-->>App: 인증 토큰 반환
    App->>DB: 상품 조회 쿼리
    DB-->>App: 조회 결과 반환
    App-->>User: 응답 반환
```

공식문서 참고

- **Azure Architecture Center**

- <https://learn.microsoft.com/en-us/azure/architecture/browse/>
- 공식 참조 아키텍처 모음

- **Azure 애플리케이션을 위한 10가지 설계 원칙**

- <https://learn.microsoft.com/en-us/azure/architecture/guide/design-principles/>
- 설계 문서 작성의 기반 원칙

- **Mermaid 공식 문서**

- <https://mermaid.js.org>
- 다이어그램 문법 레퍼런스

실습 Lab

• 실습 시나리오

- 로그인 → 상품 조회 → 주문 생성 Use Case에 대한 Sequence Diagram을 Mermaid-AI로 생성한다.

• Gen AI 프롬프트 예시

- “다음 Use Case를 Mermaid sequenceDiagram 문법으로 작성해줘: 사용자가 Application Gateway를 통해 로그인 요청을 보내고, App Service가 Azure AD B2C로 인증을 위임하며, 인증 성공 후 상품 조회 API를 호출하고 Azure SQL Database에서 데이터를 조회한 뒤 응답을 반환한다.”

• 점검 포인트

- 생성된 다이어그램에 모든 컴포넌트(Application Gateway, App Service, Azure AD B2C, SQL DB)가 순서대로 포함되어 있는지 확인
- 호출 방향(요청/응답)이 정확히 표현되었는지 검토
- 다이어그램이 실제 인증/조회 흐름과 일치하는지 검증

핵심 요약

- HLD: 시스템 전체 구조를 상위 수준으로 설명하는 핵심 문서
- Infra/Sequence Diagram: 구조와 흐름을 각각 시각화하는 두 축
- Mermaid-AI: 텍스트를 다이어그램으로 자동 변환, 버전 관리 용이
- Use Case 기반 생성: 시나리오 서술만으로 다이어그램 초안 확보
- 문서 통합: 생성된 다이어그램은 반드시 HLD에 통합하고 검토

Chapter 2. AI 기반 아키텍처 검증·문서화·운영

Section 3. 운영·모니터링 구조 설계

학습 목표

- Azure Monitor, Log Analytics, Application Insights의 구성 원리를 이해한다.
- 운영 기준 KPI를 선정하고 설계할 수 있다.
- AI를 활용해 운영 정책을 자동 추천받을 수 있다.

Azure Monitor/Log Analytics/Application Insights 구성 원리 — 개념

- Azure Monitor는 메트릭·로그를 수집하는 통합 플랫폼이며, Log Analytics Workspace와 Application Insights가 핵심 구성요소다.

Azure Monitor/Log Analytics/Application Insights 구성 원리 — 상세

• 핵심 구성요소

- Azure Monitor Metrics: 리소스별 실시간 성능 지표(CPU, 네트워크 등) 수집, 짧은 보존 기간
- Log Analytics Workspace: KQL(Kusto Query Language)로 로그 분석, 장기 보존 및 복합 쿼리 지원
- Application Insights: 애플리케이션 성능 모니터링(APM), 요청 추적, 예외 로깅, 종속성 맵 제공
- Diagnostic Settings: 리소스 로그를 Log Analytics/Storage/Event Hub로 라우팅하는 설정

Azure Monitor 기반 운영 모니터링 아키텍처



Azure Monitor/Log Analytics/Application Insights 구성 원리 — 사례

- 웹앱의 응답 지연이 발생하면 Application Insights의 종속성 맵으로 DB 호출 구간의 지연을 즉시 식별할 수 있다.

운영 기준 KPI 선정 실습 — 개념

- 좋은 KPI는 SLI(Service Level Indicator)에 기반하며, 목표(SLO)와 허용 오차(Error Budget)를 함께 정의한다.

운영 기준 KPI 선정 실습 — 상세

• 핵심 구성요소

- 가용성 KPI: 성공 요청 비율(예: 99.9% 이상)
- 성능 KPI: P95/P99 응답 시간(예: 500ms 이내)
- 처리량·오류율 KPI: 초당 요청 수(RPS), 큐 대기 시간, 5xx 에러 비율(예: 0.1% 미만)
- KPI별 알림 임계값을 설정하고 Azure Monitor Alert Rule로 자동화

운영 기준 KPI 선정 실습 — 실무 팁

- 모든 지표를 KPI로 삼기보다 비즈니스에 직결되는 3~5개 핵심 지표에 집중하는 것이 운영 효율을 높인다.

AI 기반 운영 정책 자동 추천 — 개념

- 서비스 특성과 트래픽 패턴을 AI에 입력하면 알림 임계값, 로그 보존 기간, 대시보드 구성안을 추천받을 수 있다.

AI 기반 운영 정책 자동 추천 — 절차

- Step 1. 서비스 특성 정리: SLA 목표, 트래픽 패턴, 장애 허용 범위를 문서화
- Step 2. AI 질의: Gen AI에 KPI 목표와 함께 알림 정책·대시보드 구성 추천 요청
- Step 3. 정책 검증: 추천된 임계값이 과거 트래픽 데이터와 부합하는지 확인
- Step 4. Azure Monitor 적용: Alert Rule, Action Group, Workbook 대시보드로 구현

Azure Monitor 데이터 보존 기간

구성요소	기본 보존 기간	최대 연장 가능 기간
Log Analytics 일반 테이블	30일	최대 2년 (Analytics 플랜)
Usage / AzureActivity 테이블	90일 (무료)	최대 2년
Application Insights	90일 (무료)	최대 2년
Azure Monitor Metrics	93일	장기 보관 시 Storage/Log Analytics 연계 필요

출처: Microsoft Learn — Manage data retention in a Log Analytics workspace

공식문서 참고

- **Azure Monitor 개요**

- <https://learn.microsoft.com/en-us/azure/azure-monitor/overview>
- 메트릭·로그 통합 플랫폼 소개

- **Log Analytics workspace 데이터 보존 관리**

- <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/data-retention-configure>
- 테이블별 보존 기간 설정

- **Get started with log queries**

- <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/get-started-queries>
- KQL 로그 쿼리 입문

실습 Lab

• 실습 시나리오

- P95 응답시간 500ms, 가용성 99.9% 목표인 API 서비스의 모니터링 정책을 Gen AI로 설계한다.

• Gen AI 프롬프트 예시

- “Azure App Service로 운영되는 API 서비스의 목표가 P95 응답시간 500ms 이내, 가용성 99.9%다. Azure Monitor 기반으로 추적해야 할 핵심 메트릭과 Alert Rule 임계값, 로그 보존 기간을 제안해줘.”

• 점검 포인트

- 제안된 메트릭이 가용성·성능 목표를 모두 커버하는지 확인
- Alert 임계값이 너무 민감하거나 둔감하지 않은지 검토
- 로그 보존 기간이 컴플라이언스 요구사항을 충족하는지 확인

핵심 요약

- Azure Monitor: 메트릭·로그 통합 관리 플랫폼
- Log Analytics/KQL: 심층 로그 분석의 핵심 도구
- Application Insights: 애플리케이션 성능 및 종속성 추적
- KPI 설계: SLI/SLO/Error Budget 기반 핵심 지표 3~5개 선정
- AI 활용: 트래픽 패턴 기반 알림·대시보드 정책 자동 추천

Chapter 2. AI 기반 아키텍처 검증·문서화·운영

Section 4. 운영 최적화 자동화

학습 목표

- 로깅 기반 장애 패턴 감지 방법을 이해한다.
- Latency 분석과 AI 기반 문제 탐지 기법을 익힌다.
- AI를 활용해 운영 개선안을 자동 생성하고 적용할 수 있다.

Logging 기반 장애 패턴 감지 — 개념

- 반복되는 로그 패턴(에러 코드, 예외 메시지)을 분석하면 장애의 근본 원인을 조기에 식별할 수 있다.

Logging 기반 장애 패턴 감지 — 상세

• 핵심 구성요소

- KQL 기반 이상 탐지: 특정 시간대 에러 로그 급증(spike)을 감지하는 쿼리 작성
- 로그 상관관계 분석: 여러 서비스의 로그를 시간 축으로 정렬해 장애 전파 경로 추적
- Azure Monitor의 Log Alert: 특정 KQL 쿼리 결과가 임계값을 초과하면 자동 알림 발송
- 대표 패턴: 특정 API 5xx 급증, DB 커넥션 타임아웃 반복, 특정 리전에서만 발생하는 오류

AI 기반 운영 최적화 자동화 루프



Logging 기반 장애 패턴 감지 — 사례

- '지난 1시간 동안 5xx 에러가 평시 대비 10배 증가'와 같은 KQL 쿼리를 스케줄링하여 조기 경보 체계를 구축한다.

Latency 분석 & AI 기반 문제 탐지 — 개념

- 응답 지연은 네트워크, 컴퓨팅, DB, 외부 종속성 등 여러 구간에서 발생할 수 있어 구간별 분해 분석이 필요하다.

Latency 분석 & AI 기반 문제 탐지 — 상세

• 핵심 구성요소

- Application Insights의 종단 간 트랜잭션 추적으로 지연 발생 구간(DB 쿼리, 외부 API 호출 등) 특정
- P50/P95/P99 지연 분포를 함께 봐야 일부 사용자에게 발생하는 꼬리 지연(Tail Latency) 문제를 놓치지 않는다
- AI 기반 이상 탐지: 과거 정상 범위를 학습해 통계적으로 벗어난 지연 패턴을 자동 플래깅(Azure Monitor 동적 임계값 등)

Latency 분석 & AI 기반 문제 탐지 — 실무 팁

- 지연 문제는 단일 원인보다 복합 원인(캐시 미스+DB 부하 동시 발생)인 경우가 많아 다각도 로 그를 함께 분석해야 한다.

개선안 자동 생성 실습 — 개념

- 장애·지연 로그 데이터를 AI에 입력하면 원인 가설과 개선 조치안을 자동으로 도출받을 수 있다
-

개선안 자동 생성 실습 — 절차

- Step 1. 로그·메트릭 수집: 장애 발생 시점 전후의 로그, 메트릭, 트레이스 데이터 정리
- Step 2. AI 분석 요청: 증상(에러율, 지연시간 변화)과 관련 로그를 입력해 원인 후보 요청
- Step 3. 가설 검증: AI가 제시한 원인 후보를 실제 대시보드 데이터와 대조하여 확인
- Step 4. 개선안 적용 및 회고: 조치 적용 후 효과를 측정하고 포스트모템(Postmortem) 문서로 정리

KQL(Kusto Query Language) 쿼리 예시

```
// 지난 1시간 5xx 에러 급증 탐지
requests
| where timestamp > ago(1h)
| where resultCode startswith "5"
| summarize ErrorCount = count() by bin(timestamp, 5m)
| where ErrorCount > 50

// P95 응답시간 추이 조회
requests
| where timestamp > ago(24h)
| summarize P95 = percentile(duration, 95) by bin(timestamp, 1h)
| render timechart
```

공식문서 참고

- **Learn common KQL operators (Tutorial)**

- <https://learn.microsoft.com/en-us/kusto/query/tutorials/learn-common-operators>
- KQL 핵심 연산자 실습

- **Log Analytics tutorial**

- <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-analytics-tutorial>
- 쿼리 작성과 분석 실습

- **Log queries in Azure Monitor 개요**

- <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-query-overview>
- 로그 쿼리 구조 이해

실습 Lab

• 실습 시나리오

- 특정 시간대 API 응답 지연이 P95 500ms에서 3초로 급증한 상황을 Gen AI로 분석한다.

• Gen AI 프롬프트 예시

- “Azure App Service 기반 API의 P95 응답시간이 평소 500ms에서 특정 30분 동안 3초로 급증했다. 같은 시간대 Azure SQL Database의 DTU 사용률이 95% 이상으로 치솟았고, 외부 결제 API 호출 실패율도 증가했다. 가능한 원인 가설과 우선 확인해야 할 지표, 단기/장기 개선안을 제안해줘.”

• 점검 포인트

- 원인 가설이 제공된 증상(DB DTU 급증, 외부 API 실패)과 논리적으로 연결되는지 확인
- 단기 조치(즉시 완화)와 장기 개선(근본 해결)이 구분되어 있는지 검토
- 개선안 적용 후 효과를 측정할 지표가 함께 제시되었는지 확인

핵심 요약

- KQL 기반 로그 분석: 장애 패턴을 조기에 탐지하는 핵심 수단
- 구간별 지연 분해: 네트워크·컴퓨팅·DB·외부 종속성 단위로 원인 특정
- Tail Latency: P95/P99까지 함께 관찰해야 놓치지 않는 문제
- AI 기반 원인 분석: 증상 입력만으로 원인 가설과 개선안 초안 확보
- 포스트모템: 개선 적용 후 회고로 재발 방지 체계화